



Universidad
Indoamérica

UNITÉ 3 · FONDEMENTS DES SYSTÈMES D'EXPLOITATION

Fichiers et systèmes de fichiers

Universidad Tecnológica Indoamérica

Laboratoire TD2

Comment accède-t-on aux fichiers? Deux niveaux

Sur le disque (persistant)

- Structure du système de fichiers: superbloc, tables d'inodes, blocs
- Inodes: les métadonnées de chaque fichier
- Répertoires: listes *nom* → *inode*

Dans le noyau (tant qu'il est ouvert)

- Table des descripteurs, une par processus
- Table des fichiers ouverts, une pour tout le système
- Table des inodes en mémoire (et le cache de pages)

Structure du système de fichiers



disque NVMe →

ext4: le groupe de blocs se répète le long de la partition (avec des copies du superbloc)

Sortie réelle

```
$ df -iT . | cut -c1-80; stat -f -c 'block size: %S bytes' .  
Filesystem      Type  Inodes   IUsed   IFree IUse% Mounted on  
/dev/nvme0n1p5 ext4 25600000 2372366 23227634   10% /  
block size: 4096 bytes
```

Un fichier = contenu + inode

- L'**inode** stocke: type et permissions, liens physiques, taille, périphérique, UID/GID, horodatages et l'emplacement des blocs
- Il ne stocke **pas** le nom: le nom est dans le répertoire
- **Correction de l'original:** `ctime` n'est pas la création mais le dernier *changement de l'inode*. La création (*Birth*) existe dans ext4 et se lit avec `statx()`.

Sortie réelle

```
$ stat notas.txt
  File: notas.txt
  size: 11          Blocks: 8          IO Block: 4096
regular file
Device: 259,5    Inode: 12190686    Links: 2
Access: (0664/-rw-rw-r--)  Uid: ( 1000/student)   Gid: (
1000/student)
Access: 2026-09-30 13:12:49.872600645 -0500
Modify: 2026-09-30 13:12:49.872600645 -0500
Change: 2026-09-30 13:12:49.879601063 -0500
 Birth: 2026-09-30 13:12:49.872600645 -0500
```

De l'inode aux blocs: pointeurs et extents

inode → extent tree (depth 1, 340 entries per block)

	logique	longueur	physique
#0	0	32 768	34 816
#1	32 768	32 768	67 681
#2	65 536	32 768	100 546
#3	98 304	32 768	133 411
#4	131 072	32 768	166 276
#5	163 840	9 111	199 141

bloc physique =
 $199\,141 + (172\,950 - 163\,840)$
= 208 251
blocs d'index lus: 1

le même fichier avec des pointeurs (ext3):

172 951 pointeurs dans 170 blocs d'index · cette position : 2 lectures d'index · avec des extents : 6 × 12 octets

676 MiB



le fichier en blocs logiques (0 ... 172 951); chaque bande est un extent

Un nom pointe vers un inode

Sortie réelle

```
$ ln notas.txt copia.txt; ln -s notas.txt atajo; ls -li notas.txt copia.txt atajo
12190687 lrwxrwxrwx 1 student student  9 Sep 30 13:12 atajo → notas.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 copia.txt
12190686 -rw-rw-r-- 2 student student 11 Sep 30 13:12 notas.txt
```

Lien physique (`link`)

Une autre entrée vers le **même** inode:
même numéro, `nlink = 2`. Pas vers des
répertoires ni entre systèmes de fichiers.

Lien symbolique (`symlink`)

Un **nouvel** inode dont le contenu est un
chemin (9 octets: `notas.txt`). Il peut rester
pendant.

Résoudre des chemins, lier et supprimer

\$ cat atajo

chemin relatif: part du répertoire courant, inode 1402

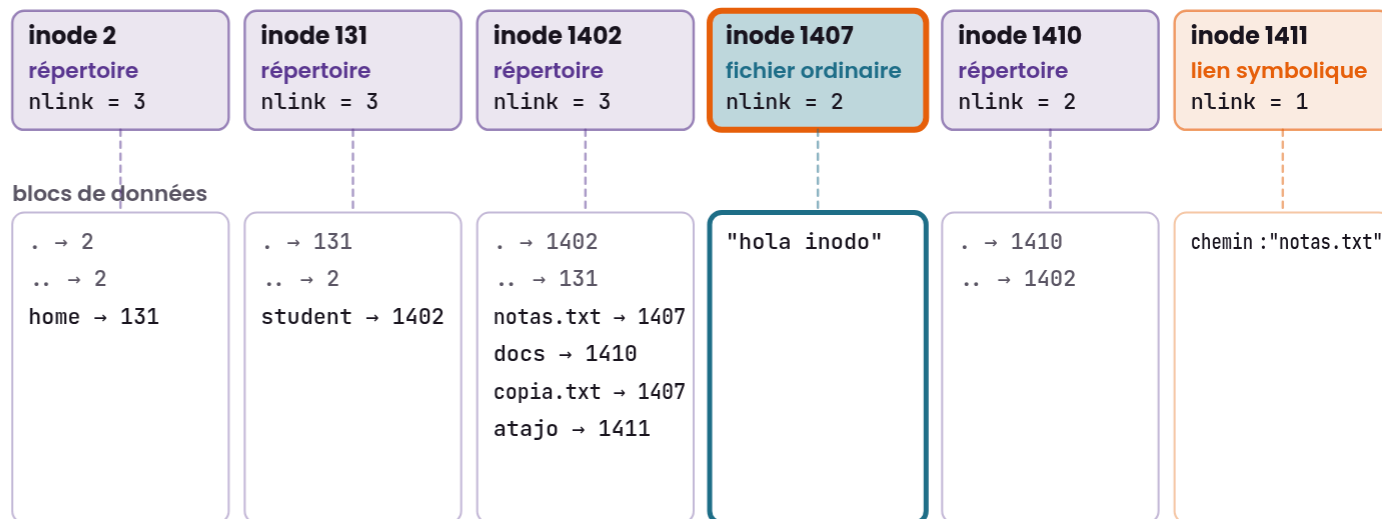
lire le rép. 1402: "atajo" → inode 1411

lien symbolique n° 1: continuer avec "notas.txt"

lire le rép. 1402: "notas.txt" → inode 1407

lire les blocs de données de l'inode 1407: "hola inodo"

table des inodes



Lire l'inode: stat, fstat, lstat

C

```
int stat(const char *path, struct stat *st);
int fstat(int fd, struct stat *st);
int lstat(const char *path, struct stat *st); /* does not follow links */

struct stat {
    dev_t      st_dev;      ino_t    st_ino;
    mode_t     st_mode;     nlink_t st_nlink;
    uid_t      st_uid;      gid_t    st_gid;
    dev_t      st_rdev;     /* device, if special file */
    off_t      st_size;     blkcnt_t st_blocks;
    blksize_t  st_blksize; /* preferred I/O block size */
    struct timespec st_atim, st_mtim, st_ctim;
};
```

- 0 en cas de succès; -1 et `errno` sinon:
- `EACCES` pas de droit `x` sur un répertoire du chemin
- `ENOENT` n'existe pas · `ENOTDIR` un composant n'est pas un répertoire
- `EL00P` plus de 40 liens symboliques
- 2026: temps en ns (`st_mtime` = `st_mtim.tv_sec`); `statx()` donne la création.

Le type est dans `st_mode` : les macros `S_IS...`

<code>S_ISREG</code>	fichier ordinaire	-
<code>S_ISDIR</code>	répertoire	d
<code>S_ISLNK</code>	lien symbolique	l
<code>S_ISCHR</code> ·	périphérique	c ·
<code>S_ISBLK</code>	caractère · bloc	b
<code>S_ISFIFO</code> ·	tube · socket	p ·
<code>S_ISSOCK</code>		s

Sortie réelle

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288
bytes, mode 0755)
```

Sortie réelle

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes,
mode 0600)
```

Sortie réelle

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584
bytes, mode 0775)
```

Permissions: `access` ... et son piège

C

```
int access(const char *path, int mode); /* R_OK W_OK X_OK F_OK */
```

- -1 s'il n'existe pas ou si la permission est refusée: `EROFS` (lecture seule), `ETXTBSY` (exécutable en cours d'utilisation)...
- Il vérifie avec l'UID *réel*, pas l'effectif

Course TOCTOU (mise à jour 2026)

Entre `access()` et `open()`, un autre processus peut remplacer le fichier par un lien vers `/etc/passwd`. La façon sûre est d'**ouvrir et gérer l'erreur** (`EACCES`), ou `openat2()` avec `RESOLVE_NO_SYMLINKS`. Le TD8 (`agentd`) utilise `realpath` et Landlock pour cette raison.

Créer, lier et supprimer des noms

C

```
int mknod(const char *path, mode_t mode, dev_t dev);
int mkfifo(const char *path, mode_t mode); /* S_IFIFO */
int link(const char *old, const char *new);
int symlink(const char *target, const char *path);
int unlink(const char *path); /* EISDIR */
int rmdir(const char *path); /* must be emp
```

- `mknod` crée n'importe quel type: le type est combiné par OU avec les permissions (`S_IFREG|S_IRUSR|S_IXUSR = S_IFREG|0500`); aujourd'hui, on ne l'utilise presque que pour les périphériques. Pour les fichiers: `open(O_CREAT)` ; pour les tubes: `mkfifo`.
- Les permissions demandées passent par le `umask` du processus (typiquement `022` ou `002`).
- `unlink` supprime une *entrée*, pas le fichier.

Ouvrir ou créer: `open`

C

```
int open(const char *path, int flags, ... /* mode_t mode */);  
int openat(int dirfd, const char *path, int flags, ...);
```

- Accès: `O_RDONLY`, `O_WRONLY`, `O_RDWR`
- Création: `O_CREAT`, `O_EXCL` (échoue s'il existe), `O_TRUNC`
- État: `O_APPEND` (chaque `write` à la fin, de façon atomique)
- Renvoie le **plus petit descripteur libre** (≥ 0) ou `-1`

Aujourd'hui aussi

- `O_CLOEXEC` : ne pas hériter du descripteur lors d'un `exec` (la glibc l'utilise toujours, voyez-le dans `strace`)
- `O_NOFOLLOW`, `O_DIRECTORY`, `O_TMPFILE`
- La glibc implémente `open` avec `openat(AT_FDCWD, ...)` ; `creat()` équivaut à `open(..., O_WRONLY|O_CREAT|O_TRUNC)`

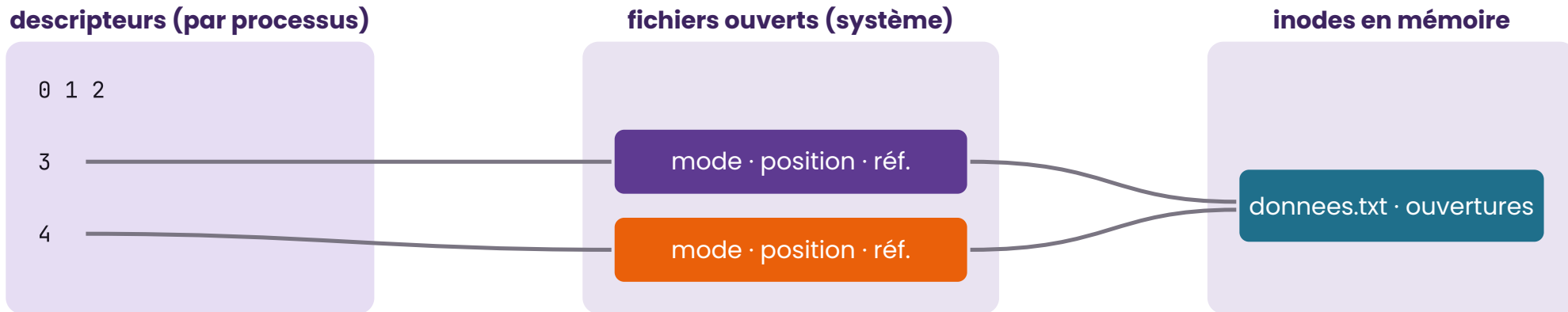
`close` : quand un fichier disparaît-il ?

Quand `nlink = 0` et qu'aucun descripteur ne l'a ouvert. D'ici là, supprimé mais vivant :

Sortie réelle

```
$ exec 3< notas.txt; rm notas.txt copia.txt; readlink /proc/$$/fd/3; cat <&3; ls -l atajo; cat atajo
~/fso/tds/tests/tmp/td2/notas.txt (deleted)
hola inodo
lrwxrwxrwx 1 student student 9 Sep 30 13:12 atajo → notas.txt
cat: atajo: No such file or directory
[exit status 1]
```

Gestion des fichiers dans le noyau: trois tables



La **position** vit dans la table du milieu: c'est pourquoi il importe de savoir qui partage chaque entrée.

Alias: deux appels `open`, `dup` et `fork`

```
fd = open("donnees.txt", O_RDONLY);
read(fd, buf, 4); write(1, buf, 4);
pid = fork();
/* child */ read(fd, buf, 4); write(1, buf, 4);
/* parent */ wait(NULL); n = read(fd, buf, 4);
read(3) utilise l'entrée A : lit "efgh" et sa position passe à 8
```

sortie standard:
"abcdefgh"

descripteurs (par processus)

parent

0	→ terminal
1	→ terminal
2	→ terminal
3	
4	

enfant

0	→ terminal
1	→ terminal
2	→ terminal
3	
4	

fichiers ouverts (système)

entrée A

O_RDONLY
offset = 8

refs = 2

inodes en mémoire (système)

donnees.txt

ouvert par 1 entrée(s)

a	b	c	d	e	f	g	h
---	---	---	---	---	---	---	---

Fonctions de base, `read` et `write` : rappel

- Inodes: `stat`, `chmod`, `chown`, `link`, `unlink` ...
- Descripteurs: `open`, `read`, `write`, `lseek`, `close`, `fcntl` ...
- `fopen` / `fread` : bibliothèque, pas des appels système (`fileno()`)

`read` peut renvoyer moins d'octets, 0 à la fin, `-1 / EINTR` ; `write` peut être partiel. L'exemple de la diapositive 51 avait un bogue: A2.4.

Parcourir les répertoires: `opendir`, `readdir`, `closedir`

C

```
DIR *opendir(const char *name);
struct dirent *readdir(DIR *dir); /* NULL at the end */
int closedir(DIR *dir);

struct dirent {
    ino_t      d_ino;      /* inode number */
    unsigned char d_type;  /* DT_REG, DT_DIR, DT_LNK... */
    char       d_name[256];
    /* d_off, d_reclen: internal */
};
```

- `readdir` renvoie les entrées dans l'ordre interne du système de fichiers (sur ext4, l'ordre d'un index haché), pas alphabétique
- Il inclut `.` et `..`
- **Mise à jour:** l'original disait « n'utilisez pas les quatre premiers champs ». Aujourd'hui `d_ino` et `d_type` sont utiles (ils évitent un `stat`), mais `d_type` peut valoir `DT_UNKNOWN` : il faut alors `lstat`.

Parcourir un répertoire: `myls -lR`

DIR *d = opendir(dir); puis:

C tds/td2/myls.c

```
struct dirent *e;
while ((e = readdir(d)) != NULL) {
    if (n == cap) {
        cap *= 2;
        names = realloc(names, cap * sizeof *names);
        exit_if(names == NULL, "realloc");
    }
    names[n] = strdup(e->d_name);
    exit_if(names[n] == NULL, "strdup");
    n++;
}
exit_if(closedir(d) == -1, "closedir");
```

Sortie réelle

```
$ ./myls -lR demo
demo:
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
-rw-rw-r--   1 student student     3 2026-09-30 13:12
a.txt
lrwxrwxrwx   1 student student     5 2026-09-30 13:12
link -> a.txt
prw-rw-r--   1 student student     0 2026-09-30 13:12
pipe
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
sub

demo/sub:
drwxrwxr-x   2 student student   4096 2026-09-30 13:12
.
drwxrwxr-x   3 student student   4096 2026-09-30 13:12
..
```

Un fichier spécial: la FIFO du chat

serveur

`read() = 0` → se termine

client 1

fermé

client 2

fermé

- le serveur revient de `open()`
- C1 ouvre la FIFO en écriture
- C2 ouvre la FIFO en écriture
- C1 ferme
- C2 ferme
- `read() = 0`: le dernier écrivain a fermé → fin de fichier

chat.fifo · tampon du noyau: 16 pages × 4 Kio = 64 Kio



↓ `read(fd, buf, 4096)` → sortie du serveur

hola a todos
buenas
¿alguien en el lab?
aquí el cliente 2
chao
me voy

`read() = 0` → EOF

Ce que l'original n'a pas eu le temps de raconter

Journalisation

ext4 et XFS écrivent d'abord dans un journal: après une coupure de courant, inutile de vérifier tout le disque.

Copie sur écriture

Btrfs et ZFS n'écrasent jamais: instantanés bon marché et sommes de contrôle de chaque bloc.

Cache et fsync

`write` s'arrête dans le cache de pages; seul `fsync` garantit que c'est arrivé sur le disque.

« Tout est fichier »: `/proc`, `/sys`, `/dev` et FUSE utilisent la même interface `open` / `read` / `write`.

TD2 corrigé + défi IA

- `filetype` avec le terminal, un répertoire, un fichier et un tube
- Le chat sur FIFO, avec deux terminaux puis trois
- `alias_open` face à `alias_dup` ; `myls -lR`

Bash

```
make -C tds && tds/tests/run.sh td2
```



Universidad
Indoamérica

À RETENIR

Trois questions

1. Où est le nom d'un fichier, et où est sa taille ?
2. Pourquoi `dup` affiche-t-il « abcdefgh » et deux appels `open` « abcdabcd » ?
3. Quand le noyau libère-t-il les blocs d'un fichier supprimé ?